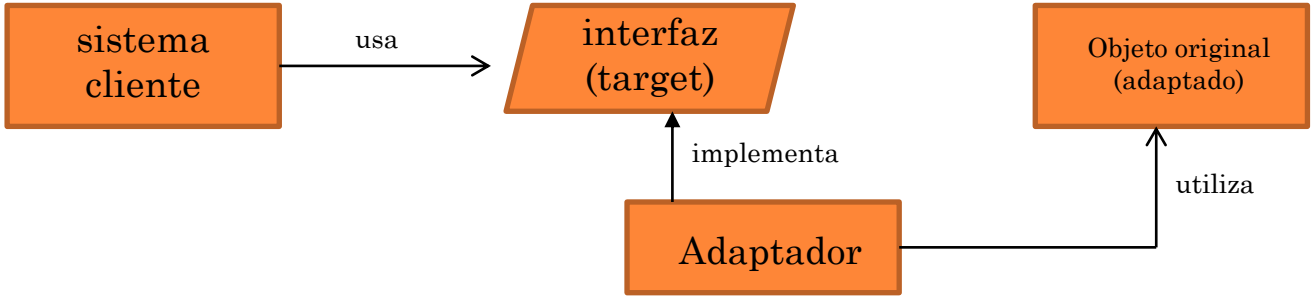
Patrones Estructurales

Patrón Adapter

- Como su nombre indica, permite adaptar una clase existente a una interfaz que no implementa, pero que es necesaria para el funcionamiento de un sistema.

- Un adaptador implementa la interfaz que el sistema debe utilizar (target), toma un objeto existente (adaptado) y "adapta" las llamadas al método de la interfaz a llamadas al objeto existente



Patrón Adapter. Ejemplo práctico

- **Supongamos la existencia de una clase que maneja textos:**

```
public class Info{
    private List<String> lst=new ArrayList<>();

    //constructor, setter y getter
    public void register(String texto){
        System.out.println(texto);
    }
    public void save(String texto){
        lst.add(texto);
    }
}
```

- **Los métodos *save()* y *register()* son específicos de esa clase. Los sistemas modernos utilizan *send()* para envío de textos:**

```
public interface Mensaje{
    void send(String texto);
}
```

Patrón Adapter. Ejemplo práctico

- **Un posible adaptador implementaría Mensaje y recibiría Info. Cada vez que se enviara un texto, tendría que almacenarlo en Info, salvarlo y registrarlo**

```
public class Adapter implements Mensaje{
    private Info info;
    public Adapter(Info info){
        this.info=info;
    }
    @Override
    public void send(String Texto){
        info.setTexto(texto);
        info.save(texto);
        info.register(texto);
    }
}
```

Patrón Adapter. Ejemplo práctico. Uso

- **Un cliente utilizaría el patrón de factoría para obtener una implementación de la interfaz y llamar a los métodos de ésta**

```
public class Cliente {
    public static void main(String[] args) {
        // Creamos la fábrica
        MensajeFactory factory = new AdapterFactory();

        // Obtenemos un objeto Mensaje a través del factory method
        Mensaje mensaje = factory.createMensaje();

        // Usamos el adaptador como si fuera un Mensaje
        mensaje.send("Hola mundo");
        mensaje.send("Patrón Adapter con Factory Method");

    }
}
```

Patrón Bridge

- El objetivo de este patrón es separar una abstracción de su implementación, a fin de que sean independientes unas de otras.

- Evita crear un sinfín de clases para cubrir todas las posibilidades de combinación de elementos: CursoOnline, MasterPresencial, etc…

- Abstracciones por un lado (Tipos de formación), implementación por otro (modalidades)

Patrón Bridge. Ejemplo práctico

- **Siguiendo el ejemplo de las formaciones, definimos una interfaz con la modalidad de formación:**

```
public interface Modalidad {
    void impartir(String nombreFormacion);
}
```

- **Y una abstracción para la formación, que vía composición hace uso de la implementación**

```
public abstract class Formacion {
    protected Modalidad modalidad;
    public Formacion(Modalidad modalidad) {
        this.modalidad = modalidad;
    }
    public abstract void formar();
}
```

Patrón Bridge. Ejemplo práctico

- **Distintas subclases de formación definirían los tipos concretos a utilizar (Curso, Master,..):**

```
public class Curso extends Formacion {
    public Curso(Modalidad modalidad) {
        super(modalidad);
    }
    @Override
    public void formar() {
        modalidad.impartir("Curso");
    }
}
public class Master extends Formacion {
    public Master(Modalidad modalidad) {
        super(modalidad);
    }
    @Override
    public void formar() {
        modalidad.impartir("Máster");
    }
}
```

Patrón Bridge. Ejemplo práctico

- **Implementaciones de Modalidad establecerían los distintos tipos de modalidades a utilizar:**

```
public class ModalidadPresencial implements Modalidad {
    @Override
    public void impartir(String nombreFormacion) {
        System.out.println(nombreFormacion +
        " se imparte de forma presencial en el aula.");
    }
}
public class ModalidadOnline implements Modalidad {
    @Override
    public void impartir(String nombreFormacion) {
        System.out.println(nombreFormacion +
        " se imparte completamente online.");
    }
}
public class ModalidadAulaVirtual implements Modalidad {
    @Override
    public void impartir(String nombreFormacion) {
        System.out.println(nombreFormacion +
        " se imparte por aula virtual teams o similar.");
    }
}
```

Patrón Bridge. Ejemplo práctico. Uso

- **Para utilizar una combinación concreta, simplemente tendríamos que instanciar la correspondiente Formacion, proporcionándole la implementación de Modalidad deseada:**

```
public class Main {
    public static void main(String[] args) {
        Formacion cursoOnline = new Curso(new ModalidadOnline());
        Formacion masterPresencial = new Master(new ModalidadPresencial());

        cursoOnline.formar();// Curso se imparte completamente online.
        masterPresencial.formar(); // Máster se imparte de forma presencial
                                   // en el aula.
      }
}
```

Patrón Decorador

- Su objetivo es añadir funcionalidad mejorada a métodos de objetos existentes, en contextos donde el objeto no puede ser modificado.
- El objeto a decorar implementa una interfaz, que también será implementada por el decorador
- El decorador también contiene como atributo al objeto a decorar
- Subclases del decorador implementan los métodos de la interfaz con la nueva funcionalidad mejorada

Patrón Decorador. Ejemplo práctico

- **Tenemos la siguiente interfaz:**

```
public interface Ordenador{
    String getDescripcion();
    double getPrecio();
}
```

- **Y el componente a decorar que la implementa con su funcionalidad original:**

```
public final class OrdenadorEstandar implements Ordenador{
    @Override
    public String getDescripcion() {
        return "Ordenador con configuración estandar";
    }
    @Override
    public double getPrecio() {
        return 500;
    }
}
```

Patrón Decorador. Ejemplo práctico

- **Definimos un decorador abstracto, cuyas subclases permitirán añadir nueva funcionalidad a los objetos:**

```
public abstract class DecoradorBase implements Ordenador {
    private Ordenador componente;
    public DecoradorBase(Ordenador componente) {
        super();
        this.componente = componente;
    }
    protected Ordenador getComponente(){return componente;}
    @Override
    public String getDescripcion() {
        return componente.getDescripcion();
    }
    @Override
    public double getPrecio() {
        return componente.getPrecio();
    }
}
```

Objeto a decorar

Patrón Decorador. Ejemplo práctico

- **Las subclases del decorador añaden funcionalidad a los métodos, haciendo uso del objeto original:**

```
public class DecoradorDisco extends DecoradorBase {

    public DecoradorDisco(Ordenador componente) {
        super(componente);

    }
    @Override
    public String getDescripcion() {
        return getComponente().getDescripcion()+" más disco SSD";
    }
    @Override
    public double getPrecio() {
        return getComponente().getPrecio()+200;
    }
}
```

Modifica la funcionalidad original de los métodos

Patrón Decorador. Ejemplo práctico. Uso

- **Una clase cliente dispone de un ordenador base y quiere "decorarlo", añadiendo la funcionalidad de disco SSD:**

```
Ordenador ordenador=new OrdenadorEstandar();
System.out.println("Sin decorar "+ordenador.getDescripcion());
System.out.println("Sin decorar "+ordenador.getPrecio());
//modifica el componente aplicando un decorador
ordenador=new DecoradorDisco(ordenador);
System.out.println("Decorado "+ordenador.getDescripcion());
System.out.println("Decorado "+ordenador.getPrecio());
```

Patrón Fachada

- Utilizado en escenarios en los que para realizar una acción de negocio hay que llamar a una serie de métodos.

- El patrón de fachada orquesta esas llamadas en un método independiente, a fin de liberar al cliente de realizar cada llamada a cada método.

Patrón Fachada. Ejemplo práctico

- **El registro de un usuario implica llamar a un método validarEmail(), después a guardarUsuario() para realizar el almacenamiento en la base de datos y finalmente a enviarEmail() para informar al usuario de su registro.**

- **En lugar de hacer las llamadas a los métodos en el cliente (por ejemplo un controller), se agrupan en un método de la lógica de negocio registrarUsuario() que orquesta las llamadas al resto de métodos:**

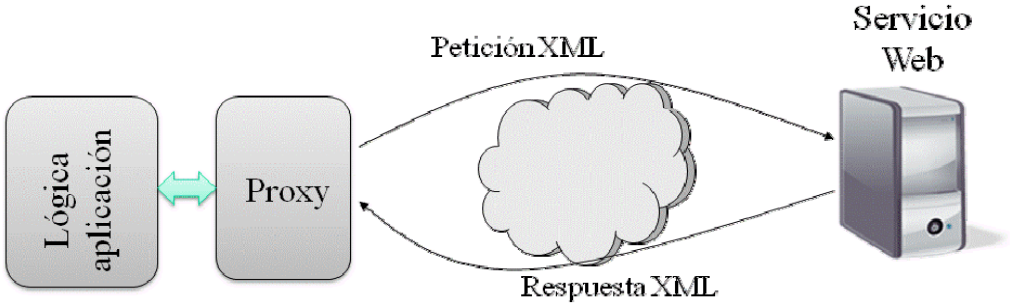
Patrón Fachada. Ejemplo práctico

```
public class UsuariosService(){
        :
        public void registrarUsuario(Usuario usuario){
                if(validarEmail(usuario.getEmail())){
                        guardarUsuario(usuario);
                        enviarEmail();
                }
                else{
                        throw new RuntimeException();
                }
        }
        :
}
```

Patrón Proxy

- Hace de intermediario entre un objeto y su cliente, cuando el uso directo del objeto resulta complejo para el cliente.

- Utilizado en contextos de acceso a objetos remoto. El proxy oculta al cliente los detalles de comunicación con el objeto remoto, exponiendo la funcionalidad al cliente a través de métodos estándares



Patrón Proxy. Ejemplo práctico

- **Un ejemplo muy utilizado de patrón proxy es el cliente feign de Spring que encapsula llamadas a servicios remotos. Se trata de un proxy autogenerado por Spring, que encapsula las llamadas a un servicio remoto.**

- **El programador simplemente define una interfaz Java con los métodos que el cliente va a utilizar.**

```
@FeignClient(name="feign",url="http://localhost:9000/")
public interface FormacionFeign {
    @GetMapping(value="alumnos",produces = "application/json")
    List<Estudiante> estudiantes();
    @PostMapping(value="alta", consumes =  "application/json")
    void altaEstudiante(@RequestBody Estudiante estudiante);
}
```

Patrón Proxy. Ejemplo práctico. Uso

- **El proxy sería utilizado en la capa de servicio, realizando las llamadas a un servicio remoto como si se tratase de un objeto local:**

```
@Service
public class FormacionServiceImpl implements FormacionService {
    @Autowired
    FormacionFeign formacionFeign;
    @Override
    public List<Estudiante> estudiantesNotaMinima(double nota) {
        return formacionFeign.estudiantes().stream()
                .filter(e->e.getCalificacion()>=nota)
                .toList();
    }
:
```

Patrón Flyweight

- Su objetivo optimizar el uso de memoria.

- La filosofía es que si una aplicación tiene que crear muchos objetos de un determinado tipo, y varios de esos objetos contienen los mismos datos, no se tengan que crear objetos con datos duplicados, sino que se reutilicen los ya existentes

Patrón Flyweight. Componentes

- **En la aplicación del patrón Flyweight distinguimos los siguientes componentes:**

  - **Flyweight: Interfaz que define los métodos de los objetos compartidos**

  - **ConcreteFlyweight: Clase que implementa la interfaz y que incluye los datos a compartir por los objetos (estado)**

  - **FlyweightFactory: Es la clase que devuelve las instancias y se encarga de la gestión de la compartición**

  - **Cliente: Utiliza el Flyweight**

Patrón Flyweight. Ejemplo práctico

- **Queremos utilizar el patrón Flyweight para generar un objeto que permita aplicar un determinado tipo de IVA a ciertos elementos.**

- **En lugar de crear objetos iguales para aplicar el mismo tipo de IVA, se mantendrá una lista de objetos IVA, uno de cada tipo, y se irán reutilizando**

- **Interfaz UsoIva:**

```
public interface UsoIVA {
    double aplicarIVA(double baseImponible);
}
```

Patrón Flyweight. Ejemplo práctico

- **Implementación concreta de la interfaz:**

```
public class UsoIVAConcreto implements UsoIVA {
    private final double porcentaje;
    public UsoIVAConcreto(double porcentaje) {
        this.porcentaje = porcentaje;
    }
    @Override
    public double aplicarIVA(double baseImponible) {
        return baseImponible*porcentaje;
    }
}
```

- **Clase de factoría que, a partir del tipo de IVA devuelve un objeto UsoIVA para ese tipo**

```
public class UsoIVAFactory {
    private final Map<String, UsoIVA> cache = new ConcurrentHashMap<>();
    public UsoIVA obtenerUso(String tipo) {
        return cache.computeIfAbsent(tipo, k -> {
            // Simular carga de config externa
            double porcentaje = switch (k) {
                case "superreducido" -> 0.04;
                case "reducido" -> 0.1;
                case "estandar" -> 0.21
                default -> 0.5;
            };
            return new UsoIVAConcreto(porcentaje);
        });
    }
}
```

Patrón Flyweight. Ejemplo práctico. Uso

- **Un cliente Flyweight podría ser un servicio de facturación que a partir de un tipo de IVA obtenga el objeto UsoIVAConcreto y lo aplique sobre una determinada base :**

```
public class FacturacionService {
    private final UsoIVAFactory usoIVAFactory;
    public FacturacionService(UsoIVAFactory usoIVAFactory) {
        this.usoIVAFactory = usoIVAFactory;
    }
    public double calcularTotal(String tipo, double base) {
        UsoIVA uso = usoIVAFactory.obtenerUso(tipo);
        return uso.aplicarIVA(base);
    }
}
```

Patrón Composite

- Se utiliza en situaciones en las que tenemos que manejar una jerarquía de objetos, y queremos tratarlos a todos de forma similar.

- En una interfaz se declaran los métodos comunes a todos los objetos, los finales que no contienen ningún otro (hoja) y los que están compuestos de otros (composite).
- Ambos tipos de objetos, hoja y composite, implementan la interfaz para así proporcionar una manera uniforme de tratarlos

Patrón Composite. Ejemplo práctico

- supongamos que tenemos una jerarquía de empleados de una empresa y queremos realizar operaciones comunes sobre todos ellos, tengan o no otros empleados a su cargo.

- La interfaz quedaría así :

```
public interface Empleado {
    String getNombre();
    String getCargo();
    void mostrarDetalles();

}
```

Patrón Composite. Ejemplo práctico

- **Un empleado simple, sin otros a su cargo, quedaría:**

```
public class EmpleadoSimple implements Empleado {
    private final String nombre;
    private final String cargo;
    public EmpleadoSimple(String nombre, String cargo) {
        this.nombre = nombre;
        this.cargo = cargo;
    }
    @Override
    public String getNombre() {
        return nombre;
    }

    @Override
    public String getCargo() {
        return cargo;
    }
    @Override
    public void mostrarDetalles() {
        System.out.println("- " + cargo + ": " + nombre);
    }
}
```

Patrón Composite. Ejemplo práctico

- **Un empleado con otros a su cargo tendría esta implementación:**

```
public class EmpleadoCompuesto implements Empleado {
    private final String nombre;
    private final String cargo;
    //lista de empleados subordinados
    private final List<Empleado> subordinados = new ArrayList<>();
    public EmpleadoCompuesto(String nombre, String cargo) {
        this.nombre = nombre;
        this.cargo = cargo;
    }
    public void agregarSubordinado(Empleado empleado) {
        subordinados.add(empleado);
    }
    @Override
    public String getNombre() {
        return nombre;
    }
    @Override
    public String getCargo() {
        return cargo;
    }
    @Override
    public void mostrarDetalles() {
        System.out.println("+ " + cargo + ": " + nombre);
        for (Empleado e : subordinados) {
            e.mostrarDetalles();
        }
    }
}
```

Patrón Composite. Ejemplo práctico. Uso

- **Un uso del patrón sería el siguiente código:**

```
public class Main {
    public static void main(String[] args) {
        EmpleadoSimple dev1 = new EmpleadoSimple("Carlos", "Desarrollador");
        EmpleadoSimple dev2 = new EmpleadoSimple("Laura", "Desarrollador");
        EmpleadoSimple rrhh = new EmpleadoSimple("Marta", "Recursos Humanos");

        EmpleadoCompuesto jefeDesarrollo = new EmpleadoCompuesto("Elena", "Jefa de Desarrollo");
        jefeDesarrollo.agregarSubordinado(dev1);
        jefeDesarrollo.agregarSubordinado(dev2);

        EmpleadoCompuesto director = new EmpleadoCompuesto("Ana", "Directora General");
        director.agregarSubordinado(jefeDesarrollo);
        director.agregarSubordinado(rrhh);

        director.mostrarDetalles();
    }
}
```

```
+ Directora General: Ana
 + Jefa de Desarrollo: Elena
  - Desarrollador: Carlos
  - Desarrollador: Laura
 - Recursos Humanos: Marta
```